An Executable Self-Justifying Axiom System in Proflog

James Torre

jpt4@proton.me

Abstract. Willard's self-justifying axiom systems (SJAS) are weak arithmetic theories able to consistently assert selected formalizations of their own consistency. Their construction is intensional: the object language, the coding of syntax, and the proof procedure jointly affect whether the self-referential consistency tableau closes. This system description reports an implementation of an $IS\#_D(\beta)$-style SJAS in Proflog, Fitting's tableau-based logic programming language. The artifact supplies a finite SJAS builder, binary U-grounding arithmetic, reflected axiom generation, inspectable formula/system/proof codes, and proof predicates that consume encoded tableau certificates. Worked examples show ordinary Proflog execution from frontend source to kernel proof traces, then show how reflected Proflog clauses become part of the finite system whose consistency sentence is generated and queried.

Keywords: self-justifying axiom systems · logic programming · semantic tableaux · proof checking · declarative programming

1 Research Objective

Willard's self-verifying and self-justifying axiom systems mark a boundary in the usual scope of Gödel's second incompleteness theorem. They do not make Peano Arithmetic prove its own consistency. They instead weaken the arithmetic language so that a carefully formulated self-consistency sentence can be added without inconsistency [3, 5].

The programming-language question is whether this construction has an executable counterpart. If self-justification depends only on an extensional set of theorems, a conventional prover for the same theorems might be enough. If it depends on the intensional behavior of deduction, then a programming language must preserve the relevant proof-tree structure, proof-code size, and syntax-generation discipline. This paper takes the second possibility seriously. It implements the ordinary-tableau $IS\#_D(\beta)$ pattern in Proflog and records where the implementation is structural, where it is merely nominal, and where further correspondence work remains.

2 Why Proflog

2.1 Self-Justification

The relevant Willard systems use a weak arithmetic base usually described as U-grounding. The constants 0 and 1, successor-like growth, and addition are available, but multiplication is not a total binary function. Instead, multiplication appears as a graph relation $mult(x, y, z)$. This choice blocks enough arithmetic strength to avoid the standard second incompleteness hypotheses while leaving enough coding capacity to discuss formulae and proofs [4, 6].

For an $IS\#_D(\beta)$ system, D is the deduction apparatus and β is a finite list of proper axioms. The self-consistency sentence asserts that no D-proof derives the contradiction code from the finite system code. The exact point at which self-justification arises is therefore not a model-theoretic slogan: it is a failure of a particular formalized proof tree to close. Tableau deduction matters because the generated tree branches on the parent formula's structure, and the U-grounding language does not supply the arithmetic strength needed to compress the pathological closure.

2.2 Proflog

Fitting's Proflog is a logic programming language whose operational semantics is semantic tableaux with equality and a Procedure Call rule [1]. To prove a query, the evaluator closes the tableau for the negated query. To refute a query, it closes the tableau for the query itself. Procedure calls open the compiled clause body, so ordinary program execution still produces tableau proof terms rather than host-language truth values.

That makes Proflog a close executable target for the most linguistically expressive SJAS variants based on semantic tableaux. It is not enough that Proflog and Willard's tableau method prove similar formulae. A proof predicate used inside an SJAS must preserve the relevant intensional measures: tree branching, closure rules, proof constructors, and encoded proof size. The implementation below is therefore organized around visible descent from source formulae to kernel proof certificates.

3 Implementation Architecture

The artifact repository is published at:

```
https://github.com/distribution-artifacts/lopstr-ppdp26
```

It contains the paper sources, Proflog implementation, tests, and worked examples cited below.

The implementation has three layers. The AST and language layer builds first-order formulae, terms, clauses, and signatures. The frontend layer gives

users a compact Clojure-readable surface syntax and expands it into AST constructors. The kernel layer runs proof search and returns proof terms with explicit constructors such as `split`, `witness`, `pos-call`, `neg-call`, `eq-step`, and closure rules.

```
surface clauses/query
  -> frontend expansion
  -> AST clauses and formulae
  -> compiled Proflog program
  -> kernel tableau search
  -> proof term / answer / unresolved status
```

Proof profiles extend the kernel with domain-specific closure behavior while keeping proof evidence visible. The SJAS profile uses this hook for U-grounding arithmetic, formula-code predicates, axiom membership, and encoded proof checking. The ordinary Proflog examples use the same kernel without the SJAS profile.

4 Worked Proflog Examples

The first examples are Fitting's even/odd program and Nim program. They are small enough to inspect but exercise quantification, equality, recursive procedure calls, and negative calls.

```
(def p1-language
  (pf/language
    (constants zero)
    (functions (s 1))
    (relations (even 1) (odd 1))))

(def p1-program
  (pf/proflog p1-language
    (|- (even x)
        (or (= x zero)
            (exists [y]
              (and (= x (s y))
                   (odd y)))))
    (|- (odd x)
        (forall [y]
          (implies (even y)
                   (!= x y))))))
```

The catalog evaluator prints the kernel-level outcome and an abbreviated proof trace. The trace below is not a side calculation: it is the certificate shape returned by the proof kernel.

```
(fitting/evaluate-case :p1-odd-1-succeeds)
;; => {:outcome :succeeds,
;;     :proof-root neg-call,
;;     :proof-steps
;;     [neg-call witness conj eq-step par-bind
;;      pos-call split free-close ... refl-close]}
```

Fitting's Nim program is represented with the one-or-two-token move relation inlined in the `win` clause:

```
(def nim-program
  (pf/proflog p1-language
```

```
    (|- (win x)
        (exists [y]
          (and (or (= x (s y))
                   (= x (s (s y))))
               (not (win y)))))))
```

The recursive negative call leaves a visible proof signature. Position 4 is a winner; position 3 is refuted under the same kernel.

```
(fitting/evaluate-case :p2-win-4-succeeds)
;; => {:outcome :succeeds,
;;     :proof-root neg-call,
;;     :proof-steps
;;     [neg-call once-univ split conj neq-close
;;      decompose args eq-bind pos-call witness ...]}

(fitting/evaluate-case :p2-win-3-fails)
;; => {:outcome :fails,
;;     :proof-root pos-call,
;;     :proof-steps
;;     [pos-call witness conj savefml split eq-step ...]}
```

Fitting warns that factoring the move test into an auxiliary relation changes operational behavior. The implementation preserves that warning: local `move` queries are classified, but `win(1)` in the factored program remains unresolved rather than being replaced by a host recurrence.

```
(fitting/evaluate-case :factored-win-1-unresolved)
;; => {:outcome :unresolved,
;;     :classification :invalid-auxiliary-relation-factoring}
```

5 Implementing $IS\#_D(\beta)$ in Proflog

5.1 System Construction

The SJAS builder accepts a proof profile, a finite β, reflected user clauses, and external user clauses. Reflected clauses are compiled as ordinary Proflog procedures and also become Group-2b axioms inside the generated finite system. External clauses are executable context only: they do not change the system code or the generated self-consistency sentence.

```
(def demo-system
  (sjas/system-source
    {:profile :willard-sjas-tableau0}
    (language
      (relations (demo 1)
                 (external-demo 1)))
    (beta
      (= 1 1))
    (reflected
      (|- (demo x)
          (= x 1)))
    (external
      (|- (external-demo x)
          (= x 0)))))

(frequencies (map :group (:axioms demo-system)))
;; => {:group-zero 2, :group-one 3, :group-two 1,
;;     :group-two-b 1, :group-three 1}
```

This distinction is important for self-reference. A source clause is part of the self-justified system only when it appears in the reflected finite basis before Group-3 generation. Merely calling an SJAS-flavored library from outside does not enlarge the consistency claim.

5.2 Codes

The implementation exposes formula, system, and proof codes as first-order terms. The default compact representation writes byte strings with generated `code-N` constructors. The stronger U-grounding representation writes the same byte strings as binary numerals over 0, 1, `dbl`, and `add`, using a sentinel so trailing zero bytes remain injective.

```
(take 12 (code/code-term-bytes (:system-code demo-system)))
;; => (31 32 2 5 25 1 0 1 25 1 0 1)

(:contradiction-code demo-system)
;; => (app code-1 (app dbl (app 1)))

(code/code-term-bytes (:contradiction-code demo-system))
;; => [2]
```

The finite symbol table is used to encode formula syntax, but the intended invariant is structural up to signature isomorphism rather than nominal. Regression tests rename reflected relation symbols, observe different byte codes, and check that the proof certificate and proof-constructor size are preserved.

5.3 Proof Predicate

The object-language predicate `tableau-proof(system,theorem,proof)` checks an encoded certificate against an encoded finite system. For the implemented certificate classes it decodes the system code, reconstructs axiom membership and reflected clauses from that code, decodes the theorem code, decodes the proof-code tree, and checks the corresponding tableau, equality, procedure-call, and axiom-citation constructors. It does not decide the predicate by consulting a host-side proof-target table.

```
(sjas/query-succeeds
  demo-system
  (pf/q (demo 1))
  {:proof-limit 1 :fuel 160})
;; first proof steps:
;; [profiled willard-sjas-tableau0 conj neg-call
;;  profiled willard-sjas-arithmetic sjas-equal ...]

(let [beta-record (first (filter #(= :group-two (:group %))
                                 (:axioms demo-system)))
      cert (sjas/proof-certificate 'sjas-axiom)]
  (query/query-succeeds
    (:program demo-system)
    (sjas/tableau-proof (:system-code demo-system)
                        (:code beta-record)
                        cert)
    1
    200))
```

```
;; first proof steps:
;; [profiled willard-sjas-tableau0
;;  profiled willard-sjas-proof-check
;;  sjas-code-bytes willard-sjas-axiom-member
;;  sjas-system-beta-axiom ... sjas-axiom]
```

The Level-1 profile similarly exposes `subst-code/2` and `subst-prf/4` for the fixed-point substitution path. Open synthesis of large public code numerals remains an operational boundary, so the focused suite tests ground and partially bound decoding paths rather than claiming unbounded code generation is cheap.

5.4 In-Principle Arithmeticization

The intended semantic object is not the fact that the host Proflog evaluator can prove a formula. It is the arithmetical relation $\mathsf{TabPrf}_\beta(s, t, p)$: the system code s decodes to a finite $IS\#_D(\beta)$ axiom basis Γ_s, the theorem code t decodes to a formula T, and the proof code p decodes to a finite closed semantic tableau for $\Gamma_s \wedge \neg T$. For the consistency sentence, t is the contradiction code, so the Group-3 assertion says that no such p exists for the system's own code.

This relation decomposes into bounded code readers, syntax checks, system-code reconstruction, axiom-membership checking, proof-tree navigation, local tableau rule checking, branch closure, substitution checking, and reflected-clause expansion. Each component is a relation over byte strings or U-grounding numerals, expressible with the profile's arithmetic vocabulary. Multiplication need not be installed as a total function; fixed-radix decoding and other arithmetic operations are used as graph relations on the concrete numerals that appear in the codes.

Soundness is then the ordinary tableau soundness induction, but applied to the decoded tree: every internal node is accepted only by a semantic-tableau rule or by a reflected procedure-call macro equivalent to expanding a decoded finite first-order clause from s, and every leaf is accepted only by a branch contradiction or by an arithmeticized profile relation. Conversely, any finite closed tableau using the admitted rules can be serialized as a proof code whose local node evidence is accepted by the relation. This is the in-principle internalization required for the SJAS claim, even when evaluating the full relation is impractical for large public numerals.

The proof-code layout is therefore required to be natural and inspectable, not byte-for-byte identical to one historical presentation. It must preserve the tree and Willard's lower-bound size discipline for semantic tableaux; theorem equivalence alone would be too weak, because a host prover could accept the same formulas with shorter or differently shaped proof objects.

6 Evaluation and Reproducibility

The repository includes the commands needed to rebuild the paper and verify the system examples:

```
make paper
lein test-proflog-fast
lein test-proflog-extended
lein test-proflog-fitting-programs
lein run -m proflog.fitting-programs p1-odd-1-succeeds p2-win-4-succeeds p2-win-3-fails
     factored-win-1-unresolved
lein test :only proflog.willard-sjas-test/sjas-system-builder-generates-groups-and-
     reflected-boundary
lein test :only proflog.willard-sjas-test/sjas-u-grounding-tableau-proof-checks-numeral-
     system-theorem-and-proof-codes
```

In the artifact repository, the focused fast suite passed 159 tests / 594 assertions in 146.92 seconds, the extended suite passed 68 tests / 203 assertions in 267.79 seconds, and the Fitting catalog gate passed 6 tests / 81 assertions in 71.02 seconds after syncing with the parent repository's current result surface. The corresponding parent Fitting run passed in 72.23 seconds.

The SJAS namespace and the full Fitting catalog are intentionally expensive. The artifact therefore provides progress-visible SJAS aliases and direct catalog-case execution so a reader can see progress one semantic test at a time before running a whole namespace. This matters for claims about proof predicates: a stalled full run gives little evidence, while focused tests identify whether code decoding, axiom membership, proof-code decoding, U-grounding arithmetic, or reflected procedure calls are responsible.

7 Limitations and Future Work

The implementation is an executable system description, not a completed formal correspondence proof between Proflog and every Willard tableau presentation. A theorem-level equivalence would be too weak for SJAS work: a host prover could prove the same theorems with shorter or differently shaped certificates and thereby change the self-referential consistency behavior. The relevant future theorem must preserve the tableau-structural invariants on which self-justification depends, while proving irrelevant differences harmless up to isomorphism.

The current proof predicate covers the certificate constructors exercised by the generated SJAS examples and regressions. Further work includes broader proof-code synthesis, stronger admissibility checks for reflected Group-2b extensions, performance improvements in recursive proof search, and a formal definition of the Proflog kernel suitable for comparing $IS\#_{D_{\text{Proflog}}}(\beta)$ with Willard's semantic-tableau $IS\#_D(\beta)$. A more speculative direction is computationalizing unclosable tableau branches as nonterminating or complexity-barrier programs.

8 Related Work

The mathematical basis is Willard's work on self-verifying and self-justifying axiom systems [3, 5] and his analysis of semantic tableaux near Robinson arithmetic [4, 6]. The programming-language basis is Fitting's Proflog [1]. The implementation uses Clojure and `core.logic` [2], but the kernel records its own proof terms rather than treating host unification as the public proof object.

9 Conclusion

Proflog gives a practical executable setting for studying Willard-style self-justification because its ordinary programming semantics is already tableau proof search. The artifact described here implements a finite $IS\#_D(\beta)$-style builder, U-grounding arithmetic, reflected Proflog extensions, inspectable syntax/proof codes, and encoded proof predicates. The result is useful both as a working logic-programming system and as a concrete testbed for the harder question: which intensional features of a deductive apparatus must be preserved for self-justification to remain meaningful?

Acknowledgements. This work was supported by the FUTO Fellowship, Summer 2025; Bloominglabs, Inc.; and Seth Frey, Department of Communication, UC Davis.

AI disclosure. Project material was created in part using Codex GPT-5.4 and GPT-5.5 (OpenAI), and Claude Opus 4.6 and 4.7 (Anthropic).

References

1. Fitting, M.: Tableaux for logic programming. Journal of Automated Reasoning **13**(2), 175–188 (1994), `https://melvinfitting.org/bookspapers/pdf/papers/LPTableaus.pdf`
2. Nolen, D., contributors: core.logic. `https://github.com/clojure/core.logic` (2026), clojure logic programming library. Accessed 2026-05-18
3. Willard, D.E.: Self-verifying axiom systems, the incompleteness theorem and related reflection principles. Journal of Symbolic Logic **66**(2), 536–596 (2001)
4. Willard, D.E.: How to extend the semantic tableaux and cut-free versions of the second incompleteness theorem almost to robinson's arithmetic q. Journal of Symbolic Logic **67**(1), 465–496 (2002)
5. Willard, D.E.: A detailed examination of methods for unifying, simplifying and extending several results about self-justifying logics (2011), `https://arxiv.org/abs/1108.6330`
6. Willard, D.E.: On the significance of self-justifying axiom systems from the perspective of analytic tableaux (2013), `https://arxiv.org/abs/1307.0150`